

AI Usage & Decision Log: LLD Practice Platform MVP

Requirement: Document 3–5 meaningful AI-assisted decisions detailing what the AI suggested, what was accepted or rejected, and the engineering rationale. ---

Decision 1: Hybrid Evaluation Pipeline Strategy (Accepted)

- **AI Suggestion:**

Initially, the AI suggested sending the raw candidate submission directly to an unconstrained LLM prompt with a prompt like *"Grade this LLD design out of 100 and give feedback"*.

- **Candidate Evaluation & Rejection of Raw Prompt:**

Unconstrained LLM calls are unreliable, non-deterministic, and prone to giving high scores (85-100) even for incomplete submissions. Furthermore, if the LLM API is down, the entire application crashes.

- **Accepted Synthesis:**

We accepted using LLM for architectural reasoning, but **constrained it with a fixed 6-axis rubric schema** and **paired it with a deterministic static analysis pre-checker**. In addition, we implemented a **Heuristic Reasoning Fallback Engine** so evaluation succeeds 100% of the time even without an active internet/API key. ---

Decision 2: Multi-Facet Submission Model vs. Pure Code Execution (Accepted)

- **AI Suggestion:**

AI suggested setting up a Docker container sandboxing environment to compile and execute candidate code (e.g. running unit tests).

- **Candidate Analysis & Decision:**

Compiling code tests algorithmic syntax, not Low-Level Design thinking. In real architectural interviews, engineers explain class responsibilities, trade-offs, and extensibility assumptions.

- **Accepted Alternative:**

We structured the `Submission` model to accept **Code + Class Responsibility Map + Design Trade-off Explanations + Visual Diagram Syntax**. This captures full evidence of

LLD reasoning with significantly lower complexity and higher signal for the learner. ---

Decision 3: Strategy Pattern for Evaluator Extensibility (Accepted)

- **AI Suggestion:**

AI proposed hardcoding evaluation logic inside a monolithic `EvaluationService.ts` method.

- **Candidate Refinement (Change Test B):**

Hardcoding evaluation methods violates the Open-Closed Principle (OCP) and makes it impossible to satisfy Change Test B (adding human review or custom static linters later).

- **Accepted Architecture:**

We created the `IEvaluator` interface and `EvaluationPipeline` composite manager. Evaluators can be registered dynamically via `registerEvaluator()`. ---

Decision 4: Asynchronous Polling Lifecycle over WebSockets (Accepted)

- **AI Suggestion:**

AI suggested setting up a Socket.io WebSocket server or Redis Pub/Sub queue to stream evaluation status to the frontend UI.

- **Candidate Decision:**

For a 2-day MVP, WebSockets and Redis introduce infrastructure overhead and state synchronization complexity without added learner value.

- **Accepted Alternative:**

We implemented an HTTP `202 Accepted` async pattern with `setImmediate` event-loop processing and a simple 700ms frontend polling mechanism. This keeps the architecture clean, lightweight, and single-port deployable.